

TypeScript Interview Notes — Zero to Senior Level

These notes cover TypeScript from fundamentals to senior-level practical usage for React, React Native, NestJS, APIs, forms, reducers, context, and production architecture.

1. What is TypeScript?

TypeScript is a superset of JavaScript that adds static typing.

JavaScript:

```
let name = "Ahtisham";  
name = 123; // JavaScript allows this
```

TypeScript:

```
let name: string = "Ahtisham";  
name = 123; // Error
```

TypeScript helps catch errors during development before runtime.

Important Point

TypeScript does not run directly in the browser. It compiles to JavaScript.

```
const name: string = "Ahtisham";
```

Compiled JavaScript:

```
const name = "Ahtisham";
```

Types are removed at runtime.

Interview Answer

TypeScript is JavaScript with static typing. It helps catch type-related errors during development, improves autocomplete, refactoring, readability, and maintainability in large projects. TypeScript compiles to JavaScript, and its types are removed at runtime.

2. Static Typing vs Dynamic Typing

JavaScript is dynamically typed

```
let value = "Hello";  
value = 123;  
value = true;
```

JavaScript allows type changes at runtime.

TypeScript is statically typed

```
let value: string = "Hello";  
value = 123; // Error
```

TypeScript checks types before runtime.

Interview Answer

JavaScript is dynamically typed, so variable types can change at runtime. TypeScript adds static typing, so types are checked during development and many bugs are caught before execution.

3. Type Inference and Explicit Types

Type Inference

TypeScript can automatically understand the type.

```
let name = "Ahtisham";  
name = 123; // Error
```

TypeScript infers `name` as `string`.

Explicit Types

```
let age: number = 25;
let isActive: boolean = true;
let userName: string = "Ahtisham";
```

Use explicit types when inference is not clear or when readability matters.

Interview Answer

Type inference means TypeScript automatically detects the type from assigned values. Explicit types are manually written when we want clarity, strict contracts, nullable state, API responses, or complex data structures.

4. Primitive Types

```
let name: string = "Ahtisham";
let age: number = 25;
let isActive: boolean = true;
let empty: null = null;
let notDefined: undefined = undefined;
const id: symbol = Symbol("id");
const bigNumber: bigint = 9007199254740991n;
```

Interview Answer

TypeScript supports JavaScript primitive types such as string, number, boolean, null, undefined, symbol, and bigint. These help make variable usage predictable and type-safe.

5. Arrays and Tuples

Arrays

```
const numbers: number[] = [1, 2, 3];
const names: string[] = ["Ali", "Ahmed", "Ahtisham"];
```

Alternative syntax:

```
const numbers: Array<number> = [1, 2, 3];
```

Array of objects:

```
const users: { id: number; name: string }[] = [  
  { id: 1, name: "Ali" },  
  { id: 2, name: "Ahmed" },  
];
```

Tuples

A tuple is an array with fixed length and fixed types.

```
const user: [number, string] = [1, "Ahtisham"];
```

Wrong:

```
const user: [number, string] = ["Ahtisham", 1]; // Error
```

Real example:

```
const coordinates: [number, number] = [31.5204, 74.3587];
```

Interview Answer

Arrays hold multiple values of the same type, while tuples define a fixed-length array with specific types at each position. Tuples are useful for coordinates, key-value pairs, and function return pairs.

6. Object Types, Optional and Readonly Properties

```
const user: {  
  id: number;  
  name: string;  
  email: string;  
} = {  
  id: 1,  
  name: "Ahtisham",  
};
```

```
    email: "test@gmail.com",  
  };
```

Optional Properties

```
const user: {  
  id: number;  
  name: string;  
  phone?: string;  
} = {  
  id: 1,  
  name: "Ahtisham",  
};
```

`phone?: string` means phone is optional.

Readonly Properties

```
const user: {  
  readonly id: number;  
  name: string;  
} = {  
  id: 1,  
  name: "Ahtisham",  
};  
  
user.name = "Ali"; // Allowed  
user.id = 2; // Error
```

Interview Answer

Object types define the structure of an object. Optional properties use `?`, and readonly properties cannot be reassigned after initialization. This is useful for IDs and immutable data.

7. Union Types

A union type means a value can be one of multiple types.

```
let value: string | number;
```

```
value = "Ahtisham";  
value = 25;  
value = true; // Error
```

Type Narrowing with Union

```
function printId(id: string | number) {  
  if (typeof id === "string") {  
    console.log(id.toUpperCase());  
  } else {  
    console.log(id.toFixed(2));  
  }  
}
```

Interview Answer

A union type allows a value to be one of multiple possible types. When using unions, we usually narrow the type before accessing type-specific methods.

8. Intersection Types

An intersection type combines multiple types into one.

```
type User = {  
  id: number;  
  name: string;  
};  
  
type Employee = {  
  employeeId: string;  
  department: string;  
};  
  
type EmployeeUser = User & Employee;  
  
const employee: EmployeeUser = {  
  id: 1,  
  name: "Ahtisham",  
  employeeId: "EMP_001",  
  department: "Engineering",  
};
```

Interview Answer

An intersection type combines multiple types into one. The final object must satisfy all combined types. Union means OR, intersection means AND.

9. Literal Types

Literal types restrict values to specific allowed values.

```
type OrderStatus = "new" | "accepted" | "delivered" | "cancelled";

function updateOrderStatus(status: OrderStatus) {
  console.log(status);
}

updateOrderStatus("new");
updateOrderStatus("completed"); // Error
```

React example:

```
type ButtonVariant = "primary" | "secondary" | "danger";

type ButtonProps = {
  title: string;
  variant: ButtonVariant;
};
```

Interview Answer

Literal types restrict a value to specific allowed values. They are useful for statuses, roles, variants, themes, screen names, and API states.

10. Type Aliases

A type alias gives a name to a type.

```
type User = {
  id: number;
  name: string;
```

```
    email: string;
};
```

Union alias:

```
type ID = string | number;
```

Function type:

```
type AddFunction = (a: number, b: number) => number;
```

Interview Answer

A type alias allows us to give a custom name to a type. It can represent primitives, objects, unions, intersections, tuples, functions, and advanced reusable types.

11. Interfaces

An interface defines the shape of an object.

```
interface User {
  id: number;
  name: string;
  email: string;
}

const user: User = {
  id: 1,
  name: "Ahtisham",
  email: "test@gmail.com",
};
```

Interface Extension

```
interface User {
  id: number;
  name: string;
}

interface Admin extends User {
```



```
permissions: string[];
}
```

Interview Answer

An interface defines the structure of an object. It is commonly used for objects, React props, API models, and class contracts. Interfaces can extend other interfaces and support declaration merging.

12. type vs interface

Both can define object shapes.

```
type UserType = {
  id: number;
  name: string;
};

interface UserInterface {
  id: number;
  name: string;
}
```

Main Differences

Interface supports declaration merging:

```
interface User {
  id: number;
}

interface User {
  name: string;
}
```

Type aliases cannot be redeclared:

```
type User = { id: number };
type User = { name: string }; // Error
```

Type aliases are more flexible:

```
type ID = string | number;
type Status = "pending" | "success" | "failed";
type Coordinates = [number, number];
type ClickHandler = (event: MouseEvent) => void;
```

Senior-Level Answer

I use `interface` when defining extendable object shapes such as props, DTOs, or domain models. I use `type` when I need unions, intersections, literal types, tuples, function types, or advanced utility types. In real projects, consistency with the codebase matters most.

13. `any`

`any` disables TypeScript type checking.

```
let value: any = "Ahtisham";

value = 123;
value = true;
value.someRandomMethod(); // TypeScript allows this
```

Problem

```
let user: any = { name: "Ahtisham" };
console.log(user.email.toLowerCase()); // Runtime crash possible
```

Interview Answer

`any` disables TypeScript type checking. It allows any value and operation, so it removes type safety. I avoid `any` unless absolutely necessary, such as migration, temporary debugging, or third-party packages without types.

14. `unknown`

`unknown` can hold any value, but we must check the type before using it.

```
let value: unknown = "Ahtisham";

value.toUpperCase(); // Error

if (typeof value === "string") {
  console.log(value.toUpperCase());
}
```

Error Handling Example

```
function handleError(error: unknown) {
  if (error instanceof Error) {
    console.log(error.message);
  } else {
    console.log("Unknown error", error);
  }
}
```

Interview Answer

`unknown` is a safer alternative to `any`. It can hold any value, but TypeScript forces us to narrow the type before using it.

15. `any` vs `unknown`

Feature	<code>any</code>	<code>unknown</code>
Can store any value	Yes	Yes
Allows any operation	Yes	No
Requires type checking	No	Yes
Type-safe	No	Yes

Interview Answer

Both `any` and `unknown` can store any value, but `any` disables type checking while `unknown` forces type narrowing before usage. Therefore, `unknown` is safer.

16. void

`void` means a function does not return a meaningful value.

```
function logMessage(message: string): void {  
  console.log(message);  
}
```

React example:

```
const handleClick = (): void => {  
  console.log("Button clicked");  
};
```

Interview Answer

`void` is used when a function does not return a meaningful value. It is common for event handlers, logging functions, and side-effect functions.

17. never

`never` means a value never exists or a function never successfully finishes.

```
function throwError(message: string): never {  
  throw new Error(message);  
}
```

Infinite loop:

```
function infiniteLoop(): never {  
  while (true) {}  
}
```

Exhaustive checking:

```
type Status = "loading" | "success" | "error";  
  
function handleStatus(status: Status) {
```

```

switch (status) {
  case "loading":
    return "Loading";
  case "success":
    return "Success";
  case "error":
    return "Error";
  default:
    const exhaustiveCheck: never = status;
    return exhaustiveCheck;
}
}

```

Interview Answer

`never` represents values that should never occur. It is used for functions that never return and for exhaustive checking in switch statements.

18. `void` vs `never`

Type	Meaning	Example
<code>void</code>	Function finishes but returns nothing meaningful	Logging function
<code>never</code>	Function never finishes successfully	Throw error, infinite loop

Interview Answer

`void` means a function completes without returning meaningful data. `never` means a function never completes successfully or a case should never happen.

19. Type Assertions

Type assertion tells TypeScript to treat a value as a specific type.

```

const input = document.getElementById("email") as HTMLInputElement;
input.value = "test@gmail.com";

```

Another example:

```
const value: unknown = "Ahtisham";  
const length = (value as string).length;
```

Important Point

Type assertion does not convert runtime values.

```
const value = "123" as unknown as number;  
console.log(value + 1); // Not real number conversion
```

Correct conversion:

```
const value = Number("123");
```

Interview Answer

Type assertion tells TypeScript to treat a value as a specific type when we know more than TypeScript. It does not perform runtime conversion, so it should be used carefully.

20. `as const`

`as const` makes values readonly and infers the most specific literal type.

Without `as const`:

```
const user = {  
  role: "admin",  
};
```

TypeScript sees:

```
{  
  role: string;  
}
```

With `as const`:

```
const user = {  
  role: "admin",  
} as const;
```

TypeScript sees:

```
{  
  readonly role: "admin";  
}
```

Create Union from Array

```
const roles = ["admin", "vendor", "customer"] as const;  
type Role = (typeof roles)[number];
```

Result:

```
type Role = "admin" | "vendor" | "customer";
```

Interview Answer

`as const` tells TypeScript to infer the most specific literal type and makes object properties and arrays readonly. It is useful for creating literal unions from constant arrays.

21. Enum vs Union Type

Enum

```
enum UserRole {  
  Admin = "admin",  
  Vendor = "vendor",  
  Customer = "customer",  
}  
  
const role: UserRole = UserRole.Admin;
```

Enums exist at runtime and generate JavaScript output.

Union Type

```
type UserRole = "admin" | "vendor" | "customer";  
const role: UserRole = "admin";
```

Union types are type-only and removed after compilation.

Senior-Level Answer

Enums define named constants and exist at runtime, while union types are type-only and removed during compilation. For simple roles, statuses, and variants, I usually prefer literal union types because they are lightweight. I use enums when runtime named constants are needed or when the project already follows enum conventions.

22. Generics

Generics allow reusable and type-safe code.

```
function identity<T>(value: T): T {  
    return value;  
}  
  
identity<string>("Ahtisham");  
identity<number>(25);
```

TypeScript can infer the type:

```
identity("Ahtisham"); // T is string  
identity(25); // T is number
```

Why Not `any`?

Bad:

```
function identity(value: any): any {  
    return value;  
}
```

Better:


```
function identity<T>(value: T): T {  
    return value;  
}
```

Interview Answer

Generics allow us to write reusable code that works with different types while preserving type safety. They are better than `any` because they keep the relationship between input and output types.

23. Generic Functions

```
function getFirstItem<T>(items: T[]): T {  
    return items[0];  
}  
  
const firstNumber = getFirstItem([1, 2, 3]);  
const firstName = getFirstItem(["Ali", "Ahmed"]);
```

TypeScript understands:

```
firstNumber; // number  
firstName; // string
```

Interview Answer

A generic function works with different types while preserving type safety. The type is passed or inferred when the function is called.

24. Generic Interfaces and Type Aliases

```
interface ApiResponse<T> {  
    success: boolean;  
    data: T;  
    message: string;  
}
```

Usage:

```

type User = {
  id: number;
  name: string;
};

const response: ApiResponse<User> = {
  success: true,
  data: {
    id: 1,
    name: "Ahtisham",
  },
  message: "User fetched successfully",
};

```

Type alias version:

```

type ApiResponse<T> = {
  success: boolean;
  data: T;
  error?: string;
};

```

Interview Answer

A generic interface or type alias creates reusable structures where part of the type can change. A common example is `ApiResponse<T>`, where `T` is the actual data returned by the API.

25. Generic Constraints

Sometimes generics need restrictions.

```

function getLength<T extends { length: number }>(value: T): number {
  return value.length;
}

getLength("Ahtisham");
getLength([1, 2, 3]);
getLength(123); // Error

```

Object constraint:

```
function printUserName<T extends { name: string }>(user: T) {  
  console.log(user.name);  
}
```

Interview Answer

Generic constraints restrict what types can be passed to a generic. We use `extends` to ensure the generic type has certain properties before accessing them.

26. `keyof`

`keyof` creates a union of object keys.

```
type User = {  
  id: number;  
  name: string;  
  email: string;  
};  
  
type UserKeys = keyof User;
```

Result:

```
type UserKeys = "id" | "name" | "email";
```

Type-safe object access:

```
function getValue<T, K extends keyof T>(obj: T, key: K) {  
  return obj[key];  
}  
  
const user = {  
  id: 1,  
  name: "Ahtisham",  
};  
  
getValue(user, "name");  
getValue(user, "age"); // Error
```

Interview Answer

`keyof` creates a union type from the keys of an object. It is commonly used with generics to create type-safe object access functions.

27. `typeof`

In TypeScript, `typeof` can create a type from an existing variable.

```
const user = {  
  id: 1,  
  name: "Ahtisham",  
  email: "test@gmail.com",  
};  
  
type User = typeof user;
```

`keyof typeof`

```
const UserRole = {  
  Admin: "admin",  
  Vendor: "vendor",  
  Customer: "customer",  
} as const;  
  
type UserRoleKey = keyof typeof UserRole;  
type UserRoleValue = (typeof UserRole)[keyof typeof UserRole];
```

Result:

```
type UserRoleKey = "Admin" | "Vendor" | "Customer";  
type UserRoleValue = "admin" | "vendor" | "customer";
```

Interview Answer

`typeof` can derive a type from an existing variable or object. It helps avoid duplicate type definitions and keeps types in sync with constants or configuration objects.

28. Indexed Access Types

Indexed access types get the type of a specific property.

```
type User = {  
  id: number;  
  name: string;  
  address: {  
    city: string;  
    country: string;  
  };  
};  
  
type UserName = User["name"];  
type UserAddress = User["address"];  
type UserCity = User["address"]["city"];
```

Array item type:

```
const users = [  
  { id: 1, name: "Ali" },  
  { id: 2, name: "Ahmed" },  
];  
  
type User = (typeof users)[number];
```

Interview Answer

Indexed access types allow us to access the type of a property from another type using bracket notation. This helps reuse existing types instead of duplicating them.

29. Mapped Types

Mapped types create new types by looping over keys of another type.

```
type User = {  
  id: number;  
  name: string;  
  email: string;  
};
```

```
type OptionalUser = {  
  [Key in keyof User]?: User[Key];  
};
```

Custom readonly:

```
type MyReadOnly<T> = {  
  readonly [Key in keyof T]: T[Key];  
};
```

Interview Answer

Mapped types create new types by iterating over keys of an existing type. They are used to build utility types like `Partial`, `Required`, `ReadOnly`, `Pick`, and custom transformations.

30. Conditional Types and `infer`

Conditional types work like if/else in the type system.

```
type IsString<T> = T extends string ? true : false;  
  
type A = IsString<string>; // true  
type B = IsString<number>; // false
```

`infer`

`infer` extracts a type inside conditional types.

```
type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never;  
  
function getUser() {  
  return {  
    id: 1,  
    name: "Ahtisham",  
  };  
}  
  
type User = MyReturnType<typeof getUser>;
```

Interview Answer

Conditional types allow type-level if/else logic using `extends`. `infer` is used inside conditional types to extract part of a type, such as a function return type, promise resolved type, or array item type.

31. Utility Types

Utility types transform existing types.

`Partial<T>`

Makes all properties optional.

```
type User = {  
  id: number;  
  name: string;  
  email: string;  
};  
  
type UpdateUserDto = Partial<User>;
```

`Required<T>`

Makes all optional properties required.

```
type RequiredUser = Required<User>;
```

`Pick<T, K>`

Selects specific properties.

```
type PublicUser = Pick<User, "id" | "name" | "email">;
```

`Omit<T, K>`

Removes specific properties.

```
type SafeUser = Omit<User, "password">;
```

Record<K, T>

Creates typed key-value objects.

```
type Role = "admin" | "vendor" | "customer";

type RolePermissions = Record<Role, string[]>;

const permissions: RolePermissions = {
  admin: ["read", "write", "delete"],
  vendor: ["read", "write"],
  customer: ["read"],
};
```

Exclude<T, U>

Removes types from a union.

```
type Status = "idle" | "loading" | "success" | "error";
type FinalStatus = Exclude<Status, "idle" | "loading">;
```

Result:

```
type FinalStatus = "success" | "error";
```

Extract<T, U>

Keeps matching types from a union.

```
type LoadingStatus = Extract<Status, "idle" | "loading">;
```

NonNullable<T>

Removes `null` and `undefined`.


```
type UserName = string | null | undefined;
type SafeUserName = NonNullable<UserName>;
```

ReturnType<T>

Gets function return type.

```
function getUser() {
  return {
    id: 1,
    name: "Ahtisham",
  };
}

type User = ReturnType<typeof getUser>;
```

Parameters<T>

Gets function parameter types as a tuple.

```
function createUser(name: string, age: number) {}

type CreateUserParams = Parameters<typeof createUser>;
```

Result:

```
type CreateUserParams = [name: string, age: number];
```

Awaited<T>

Gets resolved value from a Promise.

```
async function fetchUser() {
  return {
    id: 1,
    name: "Ahtisham",
  };
}
```

```
type User = Awaited<ReturnType<typeof fetchUser>>;
```

Interview Answer

Utility types help reuse and transform existing types. I use `Partial` for update DTOs, `Pick` for selected fields, `Omit` for removing sensitive fields, `Record` for key-value maps, `Exclude` and `Extract` for union filtering, `NonNullable` after validation, `ReturnType` and `Parameters` for function types, and `Awaited` for async result types.

32. Custom Utility Types

```
type MyPartial<T> = {  
  [Key in keyof T]?: T[Key];  
};  
  
type MyRequired<T> = {  
  [Key in keyof T]-?: T[Key];  
};  
  
type MyReadonly<T> = {  
  readonly [Key in keyof T]: T[Key];  
};  
  
type MyPick<T, K extends keyof T> = {  
  [Key in K]: T[Key];  
};  
  
type MyOmit<T, K extends keyof T> = Pick<T, Exclude<keyof T, K>>;
```

Interview Answer

Custom utility types are usually built using mapped types, `keyof`, indexed access types, and conditional types. They help create reusable type transformations.

33. Type Narrowing

Type narrowing means reducing a broad type into a more specific type.

```
function printValue(value: string | number) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  } else {
    console.log(value.toFixed(2));
  }
}
```

Narrowing Methods

```
typeof value === "string"
value instanceof Date
"permissions" in user
status === "success"
```

Interview Answer

Type narrowing is the process of reducing a broad type into a more specific type using runtime checks like `typeof`, `instanceof`, `in`, or equality checks.

34. Type Guards and Custom Type Guards

A type guard is a condition that helps TypeScript understand a more specific type.

Custom Type Guard

```
type User = {
  id: number;
  name: string;
};

function isUser(value: unknown): value is User {
  if (typeof value !== "object" || value === null) {
    return false;
  }

  return (
    "id" in value &&
    typeof value.id === "number" &&
    "name" in value &&
    typeof value.name === "string"
  );
}
```

```
    );  
}
```

Usage:

```
function handleResponse(response: unknown) {  
  if (isUser(response)) {  
    console.log(response.name);  
  } else {  
    console.log("Invalid user data");  
  }  
}
```

Interview Answer

A custom type guard is a function that returns a type predicate like `value is User`. It performs runtime checks and tells TypeScript that the value is a specific type when the function returns true.

35. Discriminated Unions

A discriminated union is a union where each type has a common identifying field.

```
type ApiState =  
  | { status: "idle" }  
  | { status: "loading" }  
  | { status: "success"; data: string[] }  
  | { status: "error"; error: string };
```

Usage:

```
function renderState(state: ApiState) {  
  switch (state.status) {  
    case "idle":  
      return "Idle";  
    case "loading":  
      return "Loading";  
    case "success":  
      return state.data.join(", ");  
    case "error":  
      return state.error;  
  }  
}
```

```
}  
}
```

Interview Answer

A discriminated union is a union where each variant has a common field with a unique literal value, such as `status`, `type`, or `kind`. It is useful for API states, form states, payment flows, and UI rendering.

36. Exhaustive Checking with `never`

```
type Status = "loading" | "success" | "error";  
  
function renderStatus(status: Status) {  
  switch (status) {  
    case "loading":  
      return "Loading";  
    case "success":  
      return "Success";  
    case "error":  
      return "Error";  
    default:  
      const exhaustiveCheck: never = status;  
      return exhaustiveCheck;  
  }  
}
```

If a new status is added but not handled, TypeScript shows an error.

Interview Answer

Exhaustive checking ensures all possible cases of a union are handled. We commonly use `never` in the default case of a switch statement to catch missing cases during development.

37. Function Overloads

Function overloads allow one function to support multiple call signatures.

```
function formatValue(value: string): string;  
function formatValue(value: number): string;
```

```
function formatValue(value: string | number): string {
  if (typeof value === "string") {
    return value.toUpperCase();
  }

  return value.toFixed(2);
}
```

Interview Answer

Function overloads allow a function to have multiple type signatures while sharing one implementation. They are useful when a function accepts different input types and returns different output types based on the input.

38. Declaration Merging

Declaration merging means TypeScript can merge multiple declarations with the same name.

```
interface User {
  id: number;
}

interface User {
  name: string;
}

const user: User = {
  id: 1,
  name: "Ahtisham",
};
```

Real use case: extending Express request.

```
declare global {
  namespace Express {
    interface Request {
      user?: {
        id: number;
        email: string;
      };
    }
  }
}
```

```
}  
}
```

Interview Answer

Declaration merging is when TypeScript merges multiple declarations with the same name, usually interfaces. It is useful for extending existing library types.

39. Module Typing

Sometimes a package does not have TypeScript types.

```
npm install --save-dev @types/some-library
```

If no types exist:

```
// declarations.d.ts  
declare module "some-library";
```

Better:

```
declare module "some-library" {  
  export function doSomething(value: string): number;  
}
```

Interview Answer

Module typing means providing TypeScript type definitions for external modules. If a package does not include types, we can install `@types/package-name` or create a custom `.d.ts` declaration file.

40. Type-Safe API Client

```
type ApiResponse<T> = {  
  success: boolean;  
  message: string;  
  data: T;  
}
```

```

};

type User = {
  id: number;
  name: string;
  email: string;
};

async function getUsers(): Promise<ApiResponse<User[]>> {
  const response = await fetch("/api/users");

  if (!response.ok) {
    throw new Error("Failed to fetch users");
  }

  return response.json();
}

```

Generic version:

```

async function apiGet<T>(url: string): Promise<ApiResponse<T>> {
  const response = await fetch(url);

  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }

  return response.json();
}

```

Usage:

```

type Product = {
  id: number;
  title: string;
  price: number;
};

const products = await apiGet<Product[]>("/api/products");

```

Senior-Level Point

TypeScript does not validate runtime API data. For critical flows, use runtime validation with Zod, Yup, or backend DTO validation.

Interview Answer

A type-safe API client uses generic request and response types so the frontend knows the exact shape of API data. I use `ApiResponse<T>` and `PaginatedResponse<T>`, and for external data I consider runtime validation because TypeScript only checks compile-time types.

41. Type-Safe Forms

```
type LoginFormValues = {
  email: string;
  password: string;
};

type FormErrors<T> = Partial<Record<keyof T, string>>;
```

Generic update function:

```
function updateField<K extends keyof LoginFormValues>(
  key: K,
  value: LoginFormValues[K]
) {
  setForm((prev) => ({
    ...prev,
    [key]: value,
  }));
}
```

Usage:

```
updateField("email", "test@gmail.com");
updateField("password", "123456");
updateField("email", 123); // Error
updateField("username", "abc"); // Error
```

Interview Answer

Type-safe forms ensure form values, validation errors, and update handlers match the expected form structure. I use a form values type, `keyof` for field updates, and `Partial<Record<keyof FormValues, string>>` for errors.

42. Type-Safe Reducers

```
type State = {
  count: number;
};

type Action =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "reset"; payload: number };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return { count: action.payload };
    default:
      const exhaustiveCheck: never = action;
      return exhaustiveCheck;
  }
}
```

Interview Answer

A type-safe reducer uses discriminated unions for actions. Each action has a `type` and only the required payload for that action. This makes reducer logic predictable and allows exhaustive checking with `never`.

43. Type-Safe Context

```
type AuthUser = {
  id: number;
  name: string;
  email: string;
};

type AuthContextValue = {
  user: AuthUser | null;
  login: (email: string, password: string) => Promise<void>;
}
```

```

    logout: () => void;
  };

  const AuthContext = createContext<AuthContextValue | undefined>(undefined);

  function useAuth() {
    const context = useContext(AuthContext);

    if (!context) {
      throw new Error("useAuth must be used inside AuthProvider");
    }

    return context;
  }

```

Interview Answer

For type-safe context, I define a context value type, create context as `ContextType | undefined`, and expose a custom hook that throws if used outside the provider. This avoids unsafe default values and repeated null checks.

44. Type-Safe Error Handling

```

try {
  // risky code
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log(error.message);
  } else {
    console.log("Unknown error", error);
  }
}

```

Custom API error:

```

type ApiError = {
  statusCode: number;
  message: string;
  code?: string;
};

function isApiError(error: unknown): error is ApiError {

```

```

return (
  typeof error === "object" &&
  error !== null &&
  "statusCode" in error &&
  "message" in error
);
}

```

Interview Answer

I avoid assuming caught errors are always `Error` objects. I type errors as `unknown`, then narrow using `instanceof Error` or custom type guards for API errors.

45. TypeScript with React Props

```

type UserCardProps = {
  name: string;
  email?: string;
  role: "admin" | "vendor" | "customer";
};

function UserCard({ name, email, role }: UserCardProps) {
  return (
    <div>
      <h3>{name}</h3>
      {email && <p>{email}</p>}
      <p>{role}</p>
    </div>
  );
}

```

Interview Answer

In React with TypeScript, I define a props type or interface for each component. Required props are typed normally, optional props use `?`, and fixed values use literal union types.

46. Typing React Children

Use `React.ReactNode`.

```
import type { ReactNode } from "react";

type CardProps = {
  children: ReactNode;
};

function Card({ children }: CardProps) {
  return <div>{children}</div>;
}
```

Interview Answer

For typing children in React, I use `React.ReactNode` because it supports almost anything React can render, including strings, numbers, JSX, fragments, arrays, null, and undefined.

47. Typing React Events

```
const handleClick = (event: React.MouseEvent<HTMLButtonElement>) => {
  console.log(event.currentTarget);
};

const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
  console.log(event.target.value);
};

const handleSubmit = (event: React.FormEvent<HTMLFormElement>) => {
  event.preventDefault();
};
```

Common Event Types

Use Case	Type
Button click	<code>React.MouseEvent<HTMLButtonElement></code>
Input change	<code>React.ChangeEvent<HTMLInputElement></code>
Textarea change	<code>React.ChangeEvent<HTMLTextAreaElement></code>
Select change	<code>React.ChangeEvent<HTMLSelectElement></code>
Form submit	<code>React.FormEvent<HTMLFormElement></code>

Interview Answer

For React events, I use React event types such as

`React.ChangeEvent<HTMLInputElement>` for inputs,

`React.MouseEvent<HTMLButtonElement>` for buttons, and

`React.FormEvent<HTMLFormElement>` for form submission.

48. Typing `useState`

Simple inference:

```
const [count, setCount] = useState(0);
```

Nullable state:

```
type User = {  
  id: number;  
  name: string;  
};  
  
const [user, setUser] = useState<User | null>(null);
```

Array state:

```
const [users, setUsers] = useState<User[]>([]);
```

API state:

```
type UserState =  
  | { status: "idle" }  
  | { status: "loading" }  
  | { status: "success"; data: User[] }  
  | { status: "error"; error: string };  
  
const [state, setState] = useState<UserState>({ status: "idle" });
```

Interview Answer

For `useState`, I rely on inference for simple values, but explicitly type nullable state, empty arrays, API responses, and discriminated unions.

49. Typing `useRef`

DOM ref:

```
const inputRef = useRef<HTMLInputElement | null>(null);  
  
inputRef.current?.focus();
```

Timer ref:

```
const timerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
```

Mutable ref:

```
const renderCount = useRef<number>(0);  
renderCount.current += 1;
```

Interview Answer

For `useRef`, I type DOM refs as `HTMLInputElement | null`, timers as `ReturnType<typeof setTimeout> | null`, and mutable values based on the stored value. Updating refs does not trigger re-render.

50. Typing `useReducer`

```
type State = {  
  count: number;  
};  
  
type Action =  
  | { type: "increment" }  
  | { type: "decrement" }  
  | { type: "reset"; payload: number };
```

```
function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "reset":
      return { count: action.payload };
    default:
      const exhaustiveCheck: never = action;
      return exhaustiveCheck;
  }
}
```

Interview Answer

I type reducers using a `State` type and an `Action` discriminated union. This gives safe dispatching and exhaustive checking.

51. Typing Custom Hooks

```
type ApiState<T> = {
  data: T | null;
  loading: boolean;
  error: string | null;
};

function useFetch<T>(url: string): ApiState<T> {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    async function fetchData() {
      try {
        setLoading(true);
        setError(null);

        const response = await fetch(url);

        if (!response.ok) {
          throw new Error("Failed to fetch data");
        }
      }
    }
  });
}
```



```

    const json = (await response.json()) as T;
    setData(json);
  } catch (err: unknown) {
    if (err instanceof Error) {
      setError(err.message);
    } else {
      setError("Something went wrong");
    }
  } finally {
    setloading(false);
  }
}

fetchData();
}, [url]);

return { data, loading, error };
}

```

Usage:

```

type User = {
  id: number;
  name: string;
};

const { data, loading, error } = useFetch<User>("/api/users");

```

Interview Answer

For custom hooks, I type input parameters and return values clearly. If the hook works with different data types, I use generics.

52. Generic React Components

```

type ListProps<T> = {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
};

function List<T>({ items, renderItem }: ListProps<T>) {
  return <div>{items.map((item, index) => <div key={index}>{renderItem(item)}</

```

```
div>}}</div>;  
}
```

Usage:

```
type User = {  
  id: number;  
  name: string;  
};  
  
<List<User>  
  items={[{ id: 1, name: "Ahtisham" }]}  
  renderItem={(user) => <p>{user.name}</p>}  
</>;
```

Interview Answer

Generic components allow reusable components while preserving strong typing. A generic list or select can work with users, products, strings, or numbers while TypeScript still knows the exact item type.

53. Typing Render Props

```
type DataLoaderProps<T> = {  
  data: T;  
  children: (data: T) => React.ReactNode;  
};  
  
function DataLoader<T>({ data, children }: DataLoaderProps<T>) {  
  return <>{children(data)}</>;  
}
```

Usage:

```
type User = {  
  id: number;  
  name: string;  
};  
  
<DataLoader<User> data={{ id: 1, name: "Ahtisham" }}>
```

```
{(user) => <p>{user.name}</p>}  
</DataLoader>;
```

Interview Answer

Render props are typed as function props that return `React.ReactNode`. If they work with dynamic data, I use generics so the function receives the correct data type.

54. Typing HOCs

```
function withAuth<P extends object>(Component: React.ComponentType<P>) {  
  return function ProtectedComponent(props: P) {  
    const isLoggedIn = true;  
  
    if (!isLoggedIn) {  
      return <p>Please login</p>;  
    }  
  
    return <Component {...props} />;  
  };  
}
```

HOC with Injected Props

```
type WithUserProps = {  
  user: {  
    id: number;  
    name: string;  
  };  
};  
  
function withUser<P extends WithUserProps>(Component: React.ComponentType<P>) {  
  return function ComponentWithUser(props: Omit<P, keyof WithUserProps>) {  
    const user = {  
      id: 1,  
      name: "Ahtisham",  
    };  
  
    return <Component {...(props as P)} user={user} />;  
  };  
}
```

Interview Answer

HOCs receive a component and return an enhanced component. In TypeScript, I use generics like `P extends object` to preserve props. If the HOC injects props, I use `Omit` to remove injected props from the external API.

55. React.FC

This works:

```
const UserCard: React.FC<UserCardProps> = ({ name }) => {  
  return <div>{name}</div>;  
};
```

But many teams prefer direct props typing:

```
function UserCard({ name }: UserCardProps) {  
  return <div>{name}</div>;  
}
```

Senior-Level Answer

I usually avoid `React.FC` and type props directly because TypeScript can infer the return type, and direct props typing works better for generic components. But if a codebase already uses `React.FC`, I follow the existing convention.

56. TypeScript with React Native Navigation

```
type RootStackParamList = {  
  Home: undefined;  
  Profile: { userId: number };  
  ProductDetails: { productId: string; title?: string };  
};
```

Navigation prop:

```
import type { NativeStackNavigationProp } from "@react-navigation/native-stack";
```

```

type HomeScreenNavigationProp = NativeStackNavigationProp<
  RootStackParamList,
  "Home"
>;

type Props = {
  navigation: HomeScreenNavigationProp;
};

```

Route prop:

```

import type { RouteProp } from "@react-navigation/native";

type ProfileRouteProp = RouteProp<RootStackParamList, "Profile">;

type Props = {
  route: ProfileRouteProp;
};

```

Interview Answer

In React Native, I type navigation using a param list like `RootStackParamList`. Each screen defines whether params are required, optional, or undefined. Then I use `NativeStackNavigationProp` for navigation and `RouteProp` for route params.

57. React Native Nested Navigation

```

import type { NavigatorScreenParams } from "@react-navigation/native";

type AuthStackParamList = {
  Login: undefined;
  Register: undefined;
};

type MainTabParamList = {
  Home: undefined;
  Orders: undefined;
  Profile: undefined;
};

type RootStackParamList = {
  Auth: NavigatorScreenParams<AuthStackParamList>;
};

```

```
Main: NavigatorScreenParams<MainTabParamList>;
ProductDetails: { productId: string };
};
```

Interview Answer

For nested navigation, I use `NavigatorScreenParams` to type nested stacks or tabs safely.

58. React Native FlatList Typing

```
type Product = {
  id: string;
  name: string;
  price: number;
};

const renderProduct: ListRenderItem<Product> = ({ item }) => {
  return (
    <View>
      <Text>{item.name}</Text>
      <Text>{item.price}</Text>
    </View>
  );
};

<FlatList<Product>
  data={products}
  keyExtractor={(item) => item.id}
  renderItem={renderProduct}
/>;
```

Interview Answer

For `FlatList`, I type the item model and pass it to `FlatList<Product>`. I also type reusable render functions with `ListRenderItem<Product>`.

59. React Native Form Typing

```
type LoginFormValues = {
  email: string;
```

```

    password: string;
  };

  type FormErrors<T> = Partial<Record<keyof T, string>>;

  const [values, setValues] = useState<LoginFormValues>({
    email: "",
    password: "",
  });

  const [errors, setErrors] = useState<FormErrors<LoginFormValues>>({});

  function updateField<K extends keyof LoginFormValues>(
    key: K,
    value: LoginFormValues[K]
  ) {
    setValues((prev) => ({
      ...prev,
      [key]: value,
    }));
  }
}

```

Usage:

```

<TextInput
  value={values.email}
  onChangeText={(text) => updateField("email", text)}
/>

```

Interview Answer

In React Native forms, I type form values, errors, and field updates. `onChangeText` directly gives a string, and a generic `updateField` ensures valid field names and value types.

60. React Native Native Event Typing

TextInput native event:

```

import type { NativeSyntheticEvent, TextInputChangeEventData } from "react-native";

function handleChange(event: NativeSyntheticEvent<TextInputChangeEventData>) {

```

```
    console.log(event.nativeEvent.text);  
  }
```

Press event:

```
import type { GestureResponderEvent } from "react-native";  
  
function handlePress(event: GestureResponderEvent) {  
  console.log(event.nativeEvent.pageX);  
}
```

Interview Answer

In React Native, I usually use `onChangeText` for text because it directly gives a string. For native events, I use `NativeSyntheticEvent<TextInputChangeEvent>`, and for press events I use `GestureResponderEvent`.

61. React Native Permission Result Typing

```
type CameraPermission =  
  | { status: "granted" }  
  | { status: "denied"; canAskAgain: boolean }  
  | { status: "blocked"; openSettingsRequired: true }  
  | { status: "unavailable"; reason: string };  
  
function getCameraPermissionMessage(permission: CameraPermission) {  
  switch (permission.status) {  
    case "granted":  
      return "Camera is ready";  
    case "denied":  
      return permission.canAskAgain  
        ? "Please allow camera permission"  
        : "Camera permission denied";  
    case "blocked":  
      return "Please open settings and enable camera permission";  
    case "unavailable":  
      return permission.reason;  
    default:  
      const exhaustiveCheck: never = permission;  
      return exhaustiveCheck;  
  }  
}
```



```
}  
}
```

Interview Answer

For permissions, I prefer literal unions or discriminated unions such as `granted`, `denied`, `blocked`, and `unavailable`. This makes permission handling safer and ensures all cases are handled.

62. Platform-Specific Type Handling

```
type AppPlatform = "ios" | "android";  
  
const storeLabels: Record<AppPlatform, string> = {  
  ios: "App Store",  
  android: "Google Play Store",  
};
```

Platform config:

```
type PlatformConfig = {  
  ios: {  
    bundleId: string;  
    appId: string;  
  };  
  android: {  
    packageName: string;  
    playStoreUrl: string;  
  };  
};
```

Interview Answer

For platform-specific logic, I use `Platform.OS` and type-safe platform config objects. `Record<"ios" | "android", T>` ensures both platforms are handled.

63. Push Notification Payload Typing

```
type NotificationPayload =
  | {
    type: "order_update";
    orderId: string;
    status: "accepted" | "delivered" | "cancelled";
  }
  | {
    type: "chat_message";
    conversationId: string;
    senderId: string;
  }
  | {
    type: "payment_success";
    paymentId: string;
    orderId: string;
  };

function handleNotification(payload: NotificationPayload) {
  switch (payload.type) {
    case "order_update":
      return payload.orderId;
    case "chat_message":
      return payload.conversationId;
    case "payment_success":
      return payload.paymentId;
    default:
      const exhaustiveCheck: never = payload;
      return exhaustiveCheck;
  }
}
```

Interview Answer

I type push notification payloads as discriminated unions using a `type` field. Each notification type has its own required data, making notification routing safer.

64. TypeScript with NestJS — DTO Typing

DTOs define request body contracts.

```
export class CreateUserDto {
  name: string;
  email: string;
  password: string;
}
```

With validation:

```
import { IsEmail, IsString, MinLength } from "class-validator";

export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;

  @IsString()
  @MinLength(8)
  password: string;
}
```

Interview Answer

In NestJS, DTOs define the shape of incoming request data. I use class-validator decorators with ValidationPipe to enforce runtime validation because TypeScript types are removed at runtime.

65. NestJS Entity / Model Typing

Prisma example:

```
import type { User } from "@prisma/client";

async function findUser(id: number): Promise<User | null> {
  return prisma.user.findUnique({ where: { id } });
}
```

Custom response type:

```
type UserResponseDto = Omit<User, "passwordHash">;
```

Interview Answer

For model typing in NestJS, I use generated Prisma or ORM types where possible, but I avoid exposing internal models directly. I create response DTOs that omit sensitive fields like password hashes.

66. NestJS Request and Response Typing

```
type ApiResponse<T> = {  
  success: boolean;  
  message: string;  
  data: T;  
};  
  
type UserResponse = {  
  id: number;  
  name: string;  
  email: string;  
};
```

Controller:

```
@Get(":id")  
async getUser(@Param("id") id: string): Promise<ApiResponse<UserResponse>> {  
  const user = await this.userService.findById(Number(id));  
  
  return {  
    success: true,  
    message: "User fetched successfully",  
    data: user,  
  };  
}
```

Interview Answer

I type NestJS service and controller return values explicitly, especially for API responses. This keeps frontend-backend contracts clear and avoids leaking internal database fields.

67. NestJS Custom Decorator Typing

Example current user decorator:

```
import { createParamDecorator, ExecutionContext } from "@nestjs/common";

export type AuthUser = {
  id: number;
  email: string;
  role: "admin" | "vendor" | "customer";
};

export const CurrentUser = createParamDecorator(
  (_data: unknown, context: ExecutionContext): AuthUser => {
    const request = context.switchToHttp().getRequest();
    return request.user;
  }
);
```

Usage:

```
@Get("me")
getMe(@CurrentUser() user: AuthUser) {
  return user;
}
```

Interview Answer

For custom decorators, I define a clear return type, such as `AuthUser`, so controllers receive strongly typed values.

68. NestJS Guard Typing

```
import { CanActivate, ExecutionContext, Injectable } from "@nestjs/common";

@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    const user = request.user as { role: string } | undefined;
```

```

    return user?.role === "admin";
  }
}

```

Better with typed user:

```

type AuthUser = {
  id: number;
  role: "admin" | "vendor" | "customer";
};

```

Interview Answer

Guards should have typed request user objects where possible. I use role unions and typed auth payloads to avoid unsafe string checks and improve authorization safety.

69. NestJS Interceptor Typing

```

import {
  CallHandler,
  ExecutionContext,
  Injectable,
  NestInterceptor,
} from "@nestjs/common";
import { Observable, map } from "rxjs";

@Injectable()
export class ResponseInterceptor<T>
  implements NestInterceptor<T, { success: boolean; data: T }>
{
  intercept(
    context: ExecutionContext,
    next: CallHandler<T>
  ): Observable<{ success: boolean; data: T }> {
    return next.handle().pipe(
      map((data) => ({
        success: true,
        data,
      })),
    );
  }
}

```

Interview Answer

Interceptors can be typed with generics so input and output response shapes are predictable. This is useful for standard API response wrappers.

70. Prisma Typing

Prisma generates types automatically.

```
import { Prisma, User } from "@prisma/client";
```

Create input:

```
const data: Prisma.UserCreateInput = {  
  name: "Ahtisham",  
  email: "test@gmail.com",  
};
```

Select typed result:

```
const userSelect = {  
  id: true,  
  name: true,  
  email: true,  
} satisfies Prisma.UserSelect;
```

Interview Answer

With Prisma, I use generated model and input types to keep database operations type-safe. I also use `select` to avoid fetching or exposing sensitive fields.

71. Generic Pagination Response Types

```
type PaginationMeta = {  
  page: number;  
  limit: number;  
  total: number;  
  totalPages: number;
```

```
};

type PaginatedResponse<T> = {
  data: T[];
  meta: PaginationMeta;
};
```

Usage:

```
type Product = {
  id: number;
  name: string;
  price: number;
};

const response: PaginatedResponse<Product> = {
  data: [{ id: 1, name: "iPhone", price: 1000 }],
  meta: {
    page: 1,
    limit: 10,
    total: 100,
    totalPages: 10,
  },
};
```

Interview Answer

I use generic pagination response types like `PaginatedResponse<T>` so the pagination structure stays consistent while the item type changes per resource.

72. Error Response Types

```
type ErrorResponse = {
  statusCode: number;
  message: string;
  error: string;
  path?: string;
  timestamp?: string;
};
```

Discriminated result:


```
type Result<T> =  
  | { success: true; data: T }  
  | { success: false; error: ErrorResponse };
```

Interview Answer

I define consistent error response types so frontend and backend can handle errors predictably. For safer handling, I often use discriminated unions like `Result<T>`.

73. Type-Safe Configuration and Environment Variables

```
type AppConfig = {  
  port: number;  
  databaseUrl: string;  
  jwtSecret: string;  
  nodeEnv: "development" | "test" | "production";  
};
```

Validation example:

```
function getConfig(): AppConfig {  
  const port = Number(process.env.PORT || 3000);  
  
  if (!process.env.DATABASE_URL) {  
    throw new Error("DATABASE_URL is required");  
  }  
  
  if (!process.env.JWT_SECRET) {  
    throw new Error("JWT_SECRET is required");  
  }  
  
  return {  
    port,  
    databaseUrl: process.env.DATABASE_URL,  
    jwtSecret: process.env.JWT_SECRET,  
    nodeEnv: (process.env.NODE_ENV || "development") as AppConfig["nodeEnv"],  
  };  
}
```

Interview Answer

Environment variables are strings at runtime, so I type and validate them at application startup. TypeScript helps with usage, but runtime validation is required to catch missing or invalid config.

74. Runtime Validation vs Compile-Time Types

TypeScript checks compile-time types only.

This is not runtime-safe:

```
const user: User = await response.json();
```

Better with Zod:

```
import { z } from "zod";

const UserSchema = z.object({
  id: z.number(),
  name: z.string(),
});

type User = z.infer<typeof UserSchema>;

const result = UserSchema.safeParse(response);

if (result.success) {
  console.log(result.data.name);
}
```

Interview Answer

TypeScript gives compile-time safety, but it does not validate external runtime data. For API responses, environment variables, forms, webhooks, or third-party data, I use runtime validation with tools like Zod, Yup, class-validator, or backend validation pipes.

75. TypeScript Strict Mode and `tsconfig` Best Practices

Recommended options:

```
{
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noFallthroughCasesInSwitch": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  }
}
```

Interview Answer

In production TypeScript projects, I prefer enabling strict mode. Important options include `strict`, `noImplicitAny`, `strictNullChecks`, and `noFallthroughCasesInSwitch`. These make bugs easier to catch during development.

76. TypeScript Performance Issues in Large Projects

Common causes:

```
Overly complex conditional types
Deep recursive types
Large union types
Too many global type declarations
Heavy inferred types from massive objects
Slow IDE type checking
```

Solutions:

```
Simplify complex types
Break types into smaller reusable pieces
Avoid unnecessary deep recursive types
Use explicit return types for complex functions
Use project references in monorepos
Keep dependencies and TypeScript version updated
```

Interview Answer

TypeScript performance can become an issue in large projects when types are too complex. I simplify conditional or recursive types, split large types, use explicit return types where helpful, and use project references in monorepos.

77. Final Senior-Level TypeScript Summary

TypeScript is JavaScript with static typing. I use it to catch errors early, improve refactoring, create safer API contracts, and make large applications easier to maintain. I understand fundamentals like primitive types, arrays, tuples, object types, unions, intersections, interfaces, type aliases, `any`, `unknown`, `void`, `never`, assertions, and `as const`. For intermediate and advanced work, I use generics, constraints, `keyof`, `typeof`, indexed access types, mapped types, conditional types, `infer`, and utility types. In React and React Native, I type props, children, events, hooks, context, navigation params, FlatList items, forms, permissions, deep links, and notification payloads. In NestJS, I type DTOs, services, responses, decorators, guards, interceptors, Prisma models, pagination, errors, and configuration. I also know that TypeScript is compile-time only, so external data still needs runtime validation.

78. Quick Interview Practice Questions

1. What is TypeScript?
2. Why use TypeScript over JavaScript?
3. What is static typing?
4. What is type inference?
5. Difference between array and tuple?
6. What are optional properties?
7. What are readonly properties?
8. What is a union type?
9. What is an intersection type?
10. What are literal types?
11. What is a type alias?
12. What is an interface?

13. Difference between `type` and `interface`?
14. Difference between `any` and `unknown`?
15. Difference between `void` and `never`?
16. What is type assertion?
17. What does `as const` do?
18. Enum vs union type?
19. What are generics?
20. Why use generics instead of `any`?
21. What are generic constraints?
22. What is `keyof`?
23. What is `typeof` in TypeScript?
24. What are indexed access types?
25. What are mapped types?
26. What are conditional types?
27. What is `infer`?
28. What is `Partial<T>`?
29. Difference between `Pick` and `Omit`?
30. What is `Record<K, T>`?
31. Difference between `Exclude` and `Extract`?
32. What is `NonNullable<T>`?
33. What is `ReturnType<T>`?
34. What is `Parameters<T>`?
35. What is `Awaited<T>`?
36. What is type narrowing?
37. What is a type guard?
38. What is a custom type guard?
39. What is a discriminated union?
40. What is exhaustive checking?
41. What are function overloads?
42. What is declaration merging?
43. How do you type external modules?
44. How do you design a type-safe API client?
45. How do you type forms?
46. How do you type reducers?
47. How do you type React Context?
48. How do you type React props?
49. How do you type React children?
50. How do you type React events?
51. How do you type `useState`?
52. How do you type `useRef`?
53. How do you type custom hooks?
54. How do you type generic components?
55. How do you type HOCs?
56. Do you use `React.FC`?
57. How do you type React Navigation params?
58. How do you type FlatList items?
59. How do you type React Native permissions?

60. How do you type push notification payloads?
 61. How do you type NestJS DTOs?
 62. How do you type NestJS service responses?
 63. How do you type custom decorators?
 64. How do you type guards and interceptors?
 65. How do you type Prisma responses?
 66. How do you type pagination responses?
 67. How do you type environment variables?
 68. Runtime validation vs compile-time typing?
 69. What are good `tsconfig` settings?
 70. How do you handle TypeScript performance issues?
-

79. Suggested Revision Order

1. TypeScript basics and primitive types
2. Arrays, tuples, object types
3. Union, intersection, literal types
4. Type aliases and interfaces
5. `any`, `unknown`, `void`, `never`
6. Type assertions and `as const`
7. Enum vs union type
8. Generics and constraints
9. `keyof`, `typeof`, indexed access types
10. Mapped types and conditional types
11. Utility types
12. Type narrowing and guards
13. Discriminated unions and exhaustive checking
14. Function overloads and declaration merging
15. Type-safe APIs, forms, reducers, context
16. TypeScript with React
17. TypeScript with React Native
18. TypeScript with NestJS
19. Runtime validation and strict mode
20. Senior-level practice questions